

Gratitude

System Requirements Specification

Project Grat - Intranet thank-you cards

August 17, 2026

Contents

1	Purpose	3
2	System overview	3
2.1	Repository layout	3
3	Hardware requirements	3
3.1	Developer workstation	3
3.2	PostgreSQL database server	3
3.3	Application server	4
3.4	Two-host layout	5
4	Software requirements	5
4.1	Runtime	5
4.2	Local development tools	5
4.3	Compile this document	5
4.4	Application stack (locked by package files)	5
5	Network and ports	6
6	Functional requirements	6
6.1	Roles	6
6.2	Core capabilities	6
6.3	Out of scope (current release)	6
7	Non-functional requirements	6
8	Configuration requirements	7
8.1	Always required	7
8.2	Typical local defaults	7
8.3	Web build-time variables	7
8.4	SMTP (MAIL_MODE=smtp)	7
8.5	Microsoft Graph (MAIL_MODE=graph)	7
9	Browser requirements	7
10	Deployment environments	8
10.1	Local	8
10.2	Continuous integration	8
10.3	Production (two hosts)	8

11 Assumptions and constraints	8
12 Document control	9

1 Purpose

This document states the hardware, software, network, and configuration requirements needed to run, develop, test, and deploy **Gratitude** (repository `project_grat`): an intranet application for designing, browsing, and sending thank-you cards by email.

It describes the system as implemented today (development login, mock directory, mock or SMTP/Graph mail) and notes planned production substitutions (Azure AD, Microsoft Graph).

2 System overview

Gratitude is a two-process web application:

- **Web** - React 19 + Vite 8 single-page application (default development URL <http://localhost:5173>).
- **API** - Fastify 5 + Prisma 7 + PostgreSQL (default <http://localhost:3001>; health check `/health`).

Card images and compiled HTML assets are stored on the API host under `api/uploads/` and served at `/uploads/`. Mail delivery is pluggable: mock (no real send), SMTP (e.g. Gmail), or Microsoft Graph (Outlook drafts).

2.1 Repository layout

Path	Role
<code>api/</code>	HTTP API, Prisma schema, migrations, mail/auth providers
<code>web/</code>	Browser UI (compose, marketplace, designer studio, admin)
<code>.github/workflows/</code>	Continuous integration (GitHub Actions)
<code>documents/</code>	Project documentation (this specification)

3 Hardware requirements

3.1 Developer workstation

Item	Minimum
CPU	2 cores
RAM	8 GB (16 GB recommended when running Postgres, Node API, and Vite together)
Disk	2 GB free for Node modules, Postgres data, and uploaded card assets
Display	1280×720 or larger for the designer and compose layouts

3.2 PostgreSQL database server

Run PostgreSQL on its own host. The application server connects with `DATABASE_URL`; card image files are *not* stored in the database.

Sizing assumes an intranet of roughly 50 to 500 operators and thousands of sent records, not a public internet load.

Item	Requirement
OS	Linux (Ubuntu 22.04/24.04 LTS or RHEL-compatible)
PostgreSQL	14 or newer (16 recommended, same as CI)
CPU	2 vCPU minimum; 4 vCPU if many concurrent compose/admin queries
RAM	4 GB minimum; 8 GB recommended (Postgres cache + connections)
Disk	40 GB SSD minimum; 80 GB+ recommended. Use a dedicated volume for PGDATA. Database rows are small; growth is mainly audit events and sent-mail metadata.
IOPS	SSD required. Avoid spinning disks.
Network	Private LAN or VPC only. Listen on TCP 5432. Allow inbound 5432 <i>only</i> from the application server (and admin jump host). No public internet. TLS recommended for the connection.
Backup	Nightly logical dump or WAL archiving to separate storage. Retain at least 7 days.
HA (optional)	Primary + replica if the organisation requires failover; not required for a small intranet.

Create an empty database named `project_grat` (or equivalent) and a dedicated role with rights only on that database. Point the API at it:

```
DATABASE_URL="postgresql://grat_user:PASSWORD@db.internal:5432/project_grat?schema=public"
```

3.3 Application server

This host runs the Node API, serves the built web UI (or sits behind nginx), and stores uploaded card assets under `api/uploads/`.

Item	Requirement
OS	Linux (Ubuntu 22.04/24.04 LTS or RHEL-compatible)
Runtime	Node.js 20 or newer; nginx (or equivalent) for HTTPS reverse proxy and static files
CPU	2 vCPU minimum; 4 vCPU recommended when designers import Canva ZIPs and rasterize PNG snapshots
RAM	4 GB minimum; 8 GB recommended (Node process, nginx, ZIP/PDF import, html-to-image)
Disk	40 GB SSD minimum for OS + Node modules. Separate persistent volume of 50 GB+ for <code>uploads/</code> (Canva images, compiled HTML, PNG snapshots). Plan about 2 to 10 MB per imported card; 50 GB covers hundreds of cards with versions.
Network inbound	TCP 443 (HTTPS) from the intranet. Optional 80 redirect to 443. Do not expose 3001 publicly; proxy it.
Network outbound	TCP 5432 to the database host; HTTPS to SMTP (587/465) or Microsoft Graph if those mail modes are used.
Process	systemd or equivalent: <code>npx tsx src/index.ts</code> (or a built start command), restart on failure. Health check: <code>/health</code> .
TLS	Certificate on nginx. Set <code>PUBLIC_API_URL</code> and <code>VITE_API_URL</code> to the public HTTPS API origin.

Note. If `uploads/` is not on persistent disk, card images disappear on rebuild or disk replacement. Keep Postgres and uploads on different machines so a web-tier rebuild does not wipe the database.

3.4 Two-host layout

Host	Runs
DB server	PostgreSQL only. No Node, no nginx, no uploads.
App server	nginx, Node API, static web/dist, uploads/ volume.
Operator PCs	Browser only.

4 Software requirements

4.1 Runtime

Component	Version	Notes
Node.js	20 or newer	Required for API (tsx) and web (Vite)
npm	10+ (bundled with Node 20)	Install from lockfiles with npm ci in CI
PostgreSQL	14 or newer (16 used in CI)	Empty database named project_grat for local development
Git	2.x	Source control and GitHub Actions checkout

4.2 Local development tools

- A PostgreSQL client (pgAdmin 4, **psql**, or equivalent) to create the database.
- A modern browser (see Section 9).
- Optional: Canva (web) to export **Email** designs as HTML + images ZIP.

4.3 Compile this document

A $\text{\LaTeX}$ distribution with **pdflatex** (TeX Live or MiKTeX) is required only to build this PDF:

```
cd documents
pdflatex system-requirements.tex
pdflatex system-requirements.tex
```

The second pass refreshes the table of contents.

4.4 Application stack (locked by package files)

Layer	Libraries	Purpose
API	Fastify, Prisma, pg, Zod, Nodemailer, JWT	HTTP, ORM, validation, mail, sessions
Web	React, React Router, Vite, JSZip, pdf.js, html-to-image	UI, Canva ZIP import, PDF page-1 fallback, PNG snapshots
Root	concurrently	Run API and web together via npm run dev

5 Network and ports

Service	Default	Purpose
Web (Vite)	TCP 5173	Developer UI; proxies <code>/api</code> to the API
API	TCP 3001	REST API, static <code>/uploads</code> , <code>/health</code>
PostgreSQL	TCP 5432	Database
SMTP	TCP 587 (STARTTLS) or 465	Optional real send when <code>MAIL_MODE=smtp</code>

The web app calls the API using `VITE_API_URL` (default `http://localhost:3001`). The API must list the web origin in `CORS_ORIGIN`. Card images in outbound HTML must be reachable at `PUBLIC_API_URL`.

6 Functional requirements

6.1 Roles

Role	Capabilities
User	Browse shared cards, send (compose), view Sent history. Cards tab opens Browse directly.
Designer	All User capabilities plus My cards: import Canva ZIP or image, categories, visibility (private/shared), advanced compose placeholders.
Admin	All Designer capabilities plus people/roles, send summary, send settings (name titles), audit log.

6.2 Core capabilities

The system shall:

1. Authenticate operators (development login today; Azure AD planned).
2. Let designers import Canva **HTML and images** ZIP files or upload PNG/JPEG/PDF.
3. Detect merge tokens such as `{{recipientName}}` and fill them at send time.
4. Let users pick recipients from the directory (mock list today; Graph planned).
5. Create mock drafts, SMTP messages, or Outlook drafts according to `MAIL_MODE`.
6. Record sent items for the operator (admins may see organisation-wide history).

6.3 Out of scope (current release)

- Physical gifts (Gifts nav is disabled).
- Azure AD interactive login (`AUTH_MODE=azure` is reserved).
- Durable object storage (S3/Blob); uploads are local disk.

7 Non-functional requirements

- **Availability.** Split the database and application onto two hosts. Keep the app process under a supervisor and back up PostgreSQL nightly.
- **Security.** JWT sessions; change `JWT_SECRET` outside local mock use. Do not commit `api/.env`. SMTP must use an app password, not a Google account password.

- **Compatibility.** HTML cards should be designed as Canva *Email* layouts (tables). Overlay text on photos may flatten to PNG on export and can break in Outlook Desktop.
- **Quality gate.** GitHub Actions CI must typecheck the API, lint/typecheck/build the web app, and apply Prisma migrations against PostgreSQL 16.

8 Configuration requirements

Copy `api/.env.example` to `api/.env`. Required and optional variables:

8.1 Always required

Variable	Meaning
DATABASE_URL	PostgreSQL connection string (database <code>project_grat</code> locally)
JWT_SECRET	Signing key for session tokens

8.2 Typical local defaults

Variable	Default	Meaning
PORT / HOST	3001 / 0.0.0.0	API bind address
CORS_ORIGIN	<code>http://localhost:5173</code>	Allowed browser origins (comma-separated)
PUBLIC_API_URL	<code>http://localhost:3001</code>	Absolute base for image URLs in mail
AUTH_MODE	<code>dev</code>	Development login form
DIRECTORY_MODE	<code>mock</code>	In-app recipient directory
MAIL_MODE	<code>mock</code>	No real mailbox send

8.3 Web build-time variables

Variable	Meaning
VITE_API_URL	API origin baked into the frontend at build time
VITE_COMPOSE_ENABLED	<code>true</code> to show send/compose in production builds; in Vite <code>dev</code> compose is on unless set to <code>false</code>

8.4 SMTP (MAIL_MODE=smtp)

Requires `SMTP_USER`, `SMTP_PASS` (Gmail App Password), and usually `SMTP_HOST=smtp.gmail.com`, `SMTP_PORT=587`, `SMTP_FROM`, `SMTP_FROM_NAME`.

8.5 Microsoft Graph (MAIL_MODE=graph)

Requires `AZURE_TENANT_ID`, `AZURE_CLIENT_ID`, `AZURE_CLIENT_SECRET`, application permission `Mail.ReadWrite` with admin consent. Optional `GRAPH_MAILBOX_UPN` pins drafts to one mailbox.

9 Browser requirements

The UI targets current evergreen browsers:

- Google Chrome, Microsoft Edge, or Firefox (latest two major versions)
- JavaScript enabled
- Desktop viewport recommended for designer and compose

Recipients of sent mail may use Outlook Desktop, Outlook on the web, or Gmail. Outlook Desktop (Word rendering) may ignore absolute positioning and some CSS used in Canva overlays.

10 Deployment environments

10.1 Local

```
npm install
cd api && npm install && npx prisma generate
npx prisma migrate dev && npm run db:seed
cd ../web && npm install && cd ..
npm run dev
```

Web: <http://localhost:5173>. API health: <http://localhost:3001/health>.

10.2 Continuous integration

GitHub Actions workflow `.github/workflows/ci.yml` runs on every push and pull request: PostgreSQL 16 service, `prisma migrate deploy`, API typecheck, web lint, typecheck, and production build.

10.3 Production (two hosts)

Use a dedicated PostgreSQL server and a separate application server (see hardware section). Secrets stay on the hosts - not in Git.

- **DB server:** install PostgreSQL, create database and role, restrict 5432 to the app server, enable backups.
- **App server:** nginx on 443; Node API proxied from `/api` or a dedicated API hostname; static files from `web/dist`; persistent `uploads/` volume.
- On deploy: `npx prisma generate` and `prisma migrate deploy` from the app server against `DATABASE_URL`; start `npx tsx src/index.ts`; health check `/health`.
- Build the web app with `VITE_API_URL` set to the public HTTPS API origin. Set `CORS_ORIGIN` and `PUBLIC_API_URL` on the API to match.

11 Assumptions and constraints

1. Operators have intranet or VPN access in production; the product is not a public marketing site.
2. Designers export Canva **Email** HTML ZIP (not a single flattened poster) when merge fields must remain editable text.
3. Development authentication does not prove identity; it is for local and staging UX only.
4. Seed data is for empty databases; do not re-seed production casually.

12 Document control

Item	Value
System name	Gratitude (<code>project_grat</code>)
Version described	0.1.0 (application package version)
Source of truth	Repository README, <code>api/.env.example</code> , <code>package.json</code> files